

Data Storage: From Paper to Databases

1. Manual Storage (Paper Notebooks)

In the beginning, data was stored manually in notebooks:

- Each page contained a table
- Each row represented one student
- Each column represented one type of information

Example:

ID	Name	Age
1	Ahmed	20
2	Sara	21

✖ Problems:

- Data can be lost easily
- Hard to search
- Hard to update
- No backup



2. Digital Files (Excel / Google Sheets)

Later, data was stored in digital files such as:

- Excel files
- Google Sheets
- CSV files

Each file was usually created for one purpose.

Example Files:

students.xlsx

ID	Name	Class
1	Ahmed	1A
2	Sara	2B

grades.xlsx

Student_ID	Math	English
1	90	85
2	70	88

fees.xlsx

Student_ID	Paid
1	Yes
2	No

Problems:

- Files are not connected
- Manual updates are required
- High risk of inconsistency
- Difficult to manage large data
- Limited security



3. Why Not Store Data in Code (Arrays)?

Example in code:

```
let students = [  
  { id: 1, name: "Ahmed" },  
  { id: 2, name: "Sara" }  
];
```

JAVASCRIPT

✗ Problems:

- Data is lost when the program stops
- No permanent storage
- Not shared between users
- Not secure
- Hard to manage large data



4. Database Storage

A Database is a specialized system designed to store data efficiently.

✓ Advantages:

- Permanent storage
- High performance
- Strong security
- Data relationships
- Multi-user support
- Backup and recovery

Example Structure:

Database: School

Table: Students

id	name	age
1	Omar	19
2	Lina	21

س

Database Concept

الداتا بيز هي مكان منظم نقدر نخزن فيه الداتا بحيث نقدر بعد كدة :

- نضيف بيانات
- نبحث عن بيانات
- نعدل بيانات
- نحذف بيانات
- و كل ده هيكون امن جدا لان :
- مش اي حد يقدر يشوف اي حاجة في الداتا بيز
- كل حاجة بتكون مربوطة ب users معينين
- يعني لو قولنا ال user ده مينفعش يمسح داتا يقدر يضيف و يبحث و يعدل بس ببقى مش هيقدر يمسح

•

- Create <- الاضافة
- Read <- البحث
- Update <- التعديل
- Delete <- حذف

ده العمليات اللي بنطبقها على اي داتا بيز و اسمها ال CRUD Operations
ايه اللي بيساعدنا نعمل العمليات ده او نعمل database اصلا هو ال Database Management System
اختصاره DBMS

يعني ما نتعامل مع فايلات معقدة ال DBMS بيسهل علينا بيساعدنا
ببقي جواه tools نقدر نعمل بيها داتا بيز و جداول و ننفذ كل العمليات بتاعت الداتا بيز

العمليات ده بنعملها بلغة اسمها SQL اختصار Structured Query Language

في DBMSs كتير موجودة زي :

- MySQL
- SQL Server
- Oracle

احنا هنختار منهم MySQL هنشتغل عليه



Connection and Database

اول حاجة احنا بنعمل connection

ايه هو ال connection ؟

هو الطريقة اللي بنقدر نتواصل بيها مع MySQL و نطلب منه ينفذ لنا مجموعة اوامر
عشان نعمل ال connection بنحتاج شوية بيانات

Host, Port, username, password

و غالبا بيكونو بالقيم ده

```
Host = 127.0.0.1
Port = 3306
Username = root
password =
```

ليه بنعمل username و password ؟
عشان طبيعي مش هنسمح لكل الناس انها تدخل عالاداتنا بيز بتاعتنا و تشوف كل حاجة و تنفذ كل العمليات كدة
لو ده حصل فاحنا بنعرض الداتا بتاعتنا للخطر ممكن تتسرق او تتمسح كلها
فلما بنعمل ابلكيشن ب node.js مثلا و هي access عال database بتاعتنا فبنديه ال username و ال password اللي
يقدر ي access بيهم عال database

طيب بعد ما بنعمل ال connection ايه الخطوة اللي بعد كدة ؟
هنكتب اول query لينا

```
SHOW DATABASES;
```

ده هيرجعلنا كل ال databases الموجودة في ال connection بتاعنا
طبيعي لو ال connection على سيرفر جديد مش هيكون في اي databases عندك فمش هيطلع ناتج لل query ده

طب دلوقتي محتاجين بقى نعمل داتا بيز عشان نبتدي نخزن فيها الداتا
خلينا نعمل داتا بيز مثلا مدرسة
بنعملها ازاي ؟

```
CREATE DATABASE school;
```

لو عايزين نمسحها بنعمل

```
DROP DATABASE school;
```

هنحتاج نخزن مين الطلاب اللي جوا المدرسة ده
هنعملهم جدول اسمه students
الجدول عبارة عن rows و columns صفوف و اعمدة
ال columns بتكون هنخزن ايه عن ال student ده مثلا اسمه و سنه و نوعه

```
CREATE TABLE students (  
  id INT,  
  name VARCHAR(255),  
  age INT,  
  gender VARCHAR(10)  
);
```

ال id وظيفته انه يعرف كل صف في الجدول
لازم يكون موجود مينفعش صف ميكونش فيه id
و لازم ميكرررش لانه بيميز الصف
طب ممكن نعمل كدة ازاي ؟

ممکن نحت جمبه خاصية `AUTO_INCREMENT` ده معناها ان لما نيجي نضيف صفوف في الجدول ده مش هحتاج نضيف id هو هيتحت لوحده و مع كل صف هنضيفه ال id هيزيد 1 عن ال id اللي قبله

لازم عشان نستخدم `AUTO_INCREMENT` يكون ال `column PRIMARY KEY` يعني ايه `PRIMARY KEY` ؟

يعني لازم ال `column` ده القيم اللي فيه `unique` يعني مبتتكرش و في نفس الوقت مينفعش نسيبه فاضي

```
CREATE TABLE students (  
  id INT AUTO_INCREMENT,  
  name VARCHAR(255),  
  age INT,  
  gender VARCHAR(10)  
);
```

لو عايزين نمسح الجدول بنعمل

```
DROP TABLE students;
```



Data Types

String Data Types

Type	Data Length	Storage Space	Ideal Use Case
CHAR	Fixed	May waste space	Short fixed codes (e.g., gender, country code)
VARCHAR	Variable	Space efficient	Long or variable-length text (e.g., names, emails, addresses)

Numeric Data Types

Data Type	Range / Notes	Typical Use Case
INT / INTEGER	-2,147,483,648 to 2,147,483,647	IDs, ages, counters
FLOAT	Approximate decimals	Scores, measurements
DOUBLE	Approximate decimals	Precise calculations

Date and Time Data Types

Data Type	Format	Notes / Use Case
DATE	'YYYY-MM-DD'	Birthdate, anniversaries
DATETIME	'YYYY-MM-DD HH:MM:SS'	Full timestamp for events
TIMESTAMP	'YYYY-MM-DD HH:MM:SS'	Auto-update on change, UTC-based
TIME	'HH:MM:SS'	Duration, start/end times
YEAR	'YYYY'	Graduation year, manufacturing year



Default Values

بنحط و احنا بنعمل الجدول قيمة default ل column معين
بحيث لو و احنا بنحط داتا في الجدول مدخلناش قيمة لل column ده بيحط القيمة ال default تلقائي

```
CREATE TABLE students (
  id INT,
  name VARCHAR(255),
  age INT DEFAULT 6,
  gender CHAR DEFAULT 'M'
);
```



Keys & Constraints

عشان نحافظ ان البيانات اللي هتدخل الجدول تكون صحيحة بنستخدم ال constraints

Constraint	Description	Example
NOT NULL	Ensures that a column cannot contain NULL values	<code>name VARCHAR(50) NOT NULL</code>
UNIQUE	Ensures that all values in a column are different	<code>email VARCHAR(100) UNIQUE</code>
PRIMARY KEY	Combines NOT NULL and UNIQUE. Identifies each row uniquely	<code>id INT PRIMARY KEY AUTO_INCREMENT</code>
CHECK	Ensures values meet a specific condition	<code>age INT CHECK (age >= 18)</code>

٣

Inserting Data in a Table

زي ما قولنا ال columns بتكون ايه البيانات اللي هحتاج نخزنها عن ال student
ال rows بتكون هي البيانات نفسها
كل row بيمثل student واحد
ازاي بقي نضيف row في الجدول

```
INSERT INTO students (name, age, gender)
VALUES ("Shams Alalfy", 18, 'F');
```

حددنا اسماء ال columns اللي هنضيف فيها و حطينا الداتا
ينفع منكاتبش اسماء ال columns ؟
ينفع بس ساعتها هنلتزم اننا ندخل كل ال columns بالترتيب اللي حاطينه ليهم و احنا بنعمل الجدول
يعني وقتها هنضطر ندخل ال id مش هيتحط لوحده

```
INSERT INTO students
VALUES (1,"Shams Alalfy", 18, 'F');
```

طب هو ينفع نعمل كدة ؟

```
INSERT INTO students (name, age, gender)
VALUES ("Shams Alalfy", 18);
```

الصف هيتضاف عادي لان كدة كدة حاطين قيمة default لل gender فهتتضاف و خلاص صح ؟
لا مش هيحصل كدة ما دام كتبنا اسم ال column مع الاسماء يبقى لازم نديه value و الا هيطلع ايرور

🔗

Reading Data from Table

عايزين نجيب الداتا اللي حطيناها في الجدول ده نعمل ايه

```
SELECT * FROM students;
```

ده هيرجع كل ال rows اللي عندنا بكل ال columns

لو عايزين نحدد columns معينة بس هي اللي تظهر

```
SELECT name, age FROM students;
```

ده هيرجعنا كل ال rows اللي عندنا و ال columns اللي حددناها بس

🔗

Updating Table Data

عشان نعدل اي row في الجدول بنستخدم **UPDATE**

متكونة من كذا جزء

اول حاجة بنحدد الجدول اللي هنعديل عليه

بعد كدة بنحدد ال column اللي هنغيره

و بعد كدة بيكون عندنا condition عشان نحدد ايه هي ال rows اللي هتتعديل بالضبط

```
UPDATE students
SET name = "Tahany"
```

```
WHERE id = 1;
```

نقدر منحطش ال condition عادي بس وقتها هيغير كل ال rows الموجودة في الجدول و ده حاجة غريبى عشان كدة هنلاقيه بيطلعنا تحذير يسألنا متاكدين من اللي بنعمله ده ولا لا

```
UPDATE students  
SET name = "Tahany";
```



Deleting From Table

ال delete شبه ال update شوية
لو احتجنا نمسح row او مجموعة rows ب condition معين بنستخدم delete

```
DELETE FROM students  
WHERE id <= 5;
```

بردو لو استخدمناها مش غير condition هتتنفذ على كل ال rows يعني كل ال rows هتتمسح

```
DELETE FROM students;
```



Where Clause

و احنا بنعمل select للداتا كنا نقدر نحدد columns معينة نجيبها
احنا نقدر بردو نجيب rows معينة ب condition معين عن طريق **WHERE**

```
SELECT name  
FROM students  
WHERE id > 10;
```

Comparison Operators

Operator	Meaning	Example
=	Equal	age = 20
!= or <>	Not equal	subject != 'Math'
>	Greater than	salary > 5000
<	Less than	age < 30
>=	Greater than or equal	age >= 18
<=	Less than or equal	age <= 40

EX:

```
SELECT *  
FROM teachers  
WHERE age > 30;
```

Logical Operators

Operator	Meaning
AND	كل الشروط لازم تتحقق
OR	شرط واحد يكفي
NOT	عكس الشرط

EX:

```
SELECT *  
FROM teachers  
WHERE subject = 'Math' AND age > 30;
```

Special Operators

1. **BETWEEN AND:** للبحث بين قيمتين

```
WHERE age BETWEEN 25 AND 40
```

2. **IN:** بتشوف القيمة بين عدة قيم ولا لا

```
WHERE subject IN ('Math', 'Physics', 'CS')
```

3. **LIKE:** عشان تخلينا نبحت باستخدام جزء من الكلمة strings بتستخدم مع ال

% -> معناها اي عدد من الحروف

_ -> معناها حرف واحد بس

```
SELECT *  
FROM teachers  
WHERE name LIKE 'Ahmed';
```

زيها زي ال = بالظبط لاننا كتبنا الكلمة كلها

```
WHERE name LIKE 'A%'
```

معناها ان اول حرف من الاسم A و مش عارفين الباقي
يعني هيرجع اي اسم اوله حرف A

```
WHERE name LIKE '%a'
```

معناها ان اخر حرف من الاسم a و مش عارفين اللي قبلها ايه هم ولا عددهم
يعني هيرجع اي اسم اخره حرف a

```
WHERE name LIKE '%ha%'
```

معناها ان في نص الاسم ha لكن في قبلها حروف و بعدها حروف منعرفش ايه هم ولا عددهم
يبقى هيجيب اي اسم يحتوي على ha

```
WHERE name LIKE 'A_'
```

معناه ان الاسم مكون من حرفين بس
اول واحد فيهم A
يعني هيرجع كل الاسماء اللي مكونة من حرفين و اول حرف فيهم A

```
WHERE name LIKE '_a%'
```

معناه ان ثاني حرف في الاسم هيكون a
قبلها حرف واحد بس و مش عارفينه
و بعدها حروف منعرفهمش ولا نعرف اد ايه
يعني هيرجع اي اسم ثاني حرف فيه حرف a